Program Optimization

F. E. ALLEN

IBM Watson Research Center, Yorktown Heights, New York

ABSTRACT.—Machine independent and language independent methods of optimizing the execution times of compiled programs are described. The approach is based on the topological characteristics of a program. A program partitioning into "strongly connected" regions is developed which permits modular optimization. The principal methods considered are eliminating redundant instructions, folding, moving instructions from one part of the program to another, reducing the strength of operators, replacing tests, and eliminating unused definitions and the computations upon which they depend. These techniques are described in enough detail to permit an evaluation of their compile time costs and the improvements they make in the object code. The appendices contain examples of programs run on the compiler which embodies the described methods.

Introduction

As computers, languages, and programming systems increase in complexity, and variety, it becomes more and more imperative that answers be sought to the following questions. Can programs written in high level languages be compiled into object code competitive with "hand written" programs; in particular, can system's programs be written in a high level language without serious degradation in execution times? If object code can be made competitive, can the compile time expense of producing such code be minimized and controlled so that compile time vs. running time trade-offs are reasonable not only for the entire program but for selected parts of it? Moreover, are there general techniques which can be applied to a variety of high level languages and for a variety of computers by which this sort of flexibility as regards compiler and object program efficiency can be obtained?

The attempt to answer these questions motivated the study described in this paper. Machine independent and language independent methods of optimizing the execution times of compiled programs were developed. This paper describes these methods in enough detail so that some conclusions regarding compile costs can be reached. In several places in the

paper evaluations of compile time vs. object code improvements are given for specific methods and their alternatives. The approach taken permits modular optimization so that it is possible to improve only the most frequently executed parts of a program. Although an empirical measure of the improvements made in problem programs is not given, an evaluation of the efficiency of the object programs obtainable by applying the described techniques is given in various parts of the paper. The reader is encouraged to draw his own conclusions about the object code by examining the appendixes. They contain several examples of programs run on the compiler which was written as part of the study.

The principal methods of optimization considered are:

1. Eliminating redundant instructions.
2. Replacing uses of variables by their current values if they are known and performing operations involving constants. This process is termed folding.
3. Moving instructions invariant in one part of a program to another, less frequently executed, part of the program.
4. Reducing the strength of operations. Sequences of instructions involving recursively defined variables (e.g., $I = I + 1$) can sometimes be replaced by a single instruction. One of the effects of this is to transform subscript expansions to index increments or decrements.
5. Replacing tests. It frequently occurs that the programmer's tests can be transformed to effect improvements.
6. Eliminating unused definitions and the computations upon which they depend. An unused definition is a result of programmer error or carelessness in the source program; optimization may, however, transform a used definition into an unused one.

Other methods of optimization, not implemented in the compiler, are also discussed.

The methods are said to be machine independent because the improvements made to the problem program will cause it to run faster on a variety of computers. They are relatively language independent in that they are applicable to high level languages in general and the optimizing techniques do not depend upon the characteristics of any particular language.

The optimization techniques developed are based on the topology of problem programs. Any frequency information inherent in the program or supplied by the programmer is utilized but the techniques do not depend on it being available. It was felt that dependency on such information would compromise the general usefulness of the techniques.

Because of the availability of FORTRAN II programs, FORTRAN

II was chosen as the source language for the optimizing compiler. Most of the compiler itself was written in FORTRAN II in order to provide a further measure of the techniques. FORTRAN II is translated into an intermediate text which is devoid of any source language characteristics. DOs, subscript combinations, tests, etc., are expanded and expressed as basic operations.

Intermediate Text

The intermediate text has the following basic constituents:

1. A *symbol table* containing the usual information about programmer variables, constants, and, after optimization, generated names.
2. A *flow table* defining the basic blocks (see page 243) of the program and their flow relationships. The basic blocks are arbitrarily numbered according to their flow table entries and their flow relationships are expressed in terms of these numbers. Any frequency information specified by the programmer or collected from DO's is available in the flow table.
3. An *instruction table* containing the unique instructions in the program. Each instruction generally has the form

$$\alpha_i : \mathrm{ARG}_1 \ \mathrm{OP} \ \mathrm{ARG}_2$$

where α_i is both the ith entry in the instruction table and, for instructions which do not have an explicit result, the implied name of the result; OP is the operation; ARG_1 and ARG_2 are the operands.

4. A *sequence table* which defines the linear order of the instructions in a block. It contains a sequence of α's which point to the instruction table. The sequence table entries for a basic block are delineated by pseudo-operations which are pointed to by the flow table entry for the block.
5. Some subsidiary tables are kept for functions, etc.

In the *basic operations*, +, −, *, /, **, etc., the arguments have one of the following forms:

1. If the argument is an unsubscripted variable or constant it is expressed as a pointer to the entry for that variable in the symbol table.
2. If the argument is the result of a previous instruction it is an α pointer to the instruction table entry for the instruction generating the result.

3. If the argument is a subscripted variable then it has 3 subparts:
   (a) The pointer to the symbol table entry for that variable.
   (b) An α pointer to the last instruction in the computation of the expanded subscript value or, if no computation is involved, the subscript variable expressed as a pointer to the symbol table.
   (c) The constant part of the subscript computation.

Defining operations are those which assign a value to a variable (e.g., A = 5, and B = α_6). The first argument in a defining operation is the variable and must, therefore, be of form 1 or 3 above. The second argument can have any of the 3 forms.

The control operations (derived from GO TO's, IF's, DO's, etc.) generally contain pointers to the flow table and, for test dependent transfers, the instruction setting up the test condition.

The following example shows in symbolic form the sequence and instruction table entries from the translation of the FORTRAN statements

```
      DO 2 I = 1, 10
2     A(I, I) = A(I, I) + 5.0
      STOP
```

for which A is a 10 × 10 array.

Entries	*Sequence Table*	*Instruction Table*
1	for Block 1	I = 1
2	α_1	TRA 2
3	α_2	I* 10
4	for Block 2	α_3 + I
5	α_3	A(α_4, −11) + 5.0
6	α_4	A(α_4, −11) = α_5
7	α_5	I + 1
8	α_3	I = α_7
9	α_4	COMPARE I AGAINST 10
10	α_6	TRA on α_9 TO 2, 2, 3
11	α_7	STOP
12	α_8	
13	α_9	
14	α_{10}	
15	for Block 3	
16	α_{11}	

The α's in the example are integer pointers.

The intermediate text has several features which are exploited by the optimization procedures.

1. Formal identities are exposed. When eliminating redundant instructions, identical pointers in the sequence table are used to trigger the search for redundancies.
2. An α used in an instruction is always defined by an instruction which precedes it in the same block.
3. An α defined by an instruction is always used later in the same block.

The optimizing procedures permute the intermediate text but before completing the optimization of any set of blocks care is taken to assure that the above 3 features still exist. This often means that generated names must be introduced to prevent trans-block uses of α pointers.

The examples in this paper may sometimes contain FORTRAN statements. This is done to expedite expression of the problem since the α notation is somewhat cumbersome for exposition.

Program Topology

A *basic block* is a linear sequence of instructions having one entry point—the first instruction executed—and one exit point—the last instruction executed. It may of course have many predecessors and many successors and may even be its own successor. Program entry blocks may have no predecessors and program terminating blocks never have successors.

Using the blocks as basic units, the flow of the program may be represented by a *directed graph* in which a node represents a basic block and an edge represents a flow path. The example in Appendix I shows the flow of two versions of a matrix multiply problem represented by a directed graph.

Several concepts from directed graph theory are of interest in the analysis of program flow. A *strongly connected region* of a directed graph is a set of nodes in which there is a flow path from any node in the set to any other node (see ref. [1]). Every node in a strongly connected region is its own predecessor and its own successor. This concept forms the basis for the flow analysis done by the optimizer.

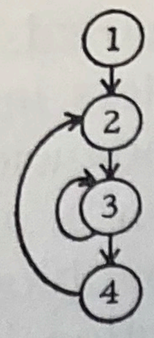
The flow of a program is examined and a list, R, of regions, $R_1, R_2, \ldots, R_n$, is constructed which has the following properties:

1. R_i is strongly connected. Each region on the list is composed of a set of blocks whose predecessor–successor relationships make them strongly connected.
2. $R_i \neq R_j$.
3. For $i < j$ either

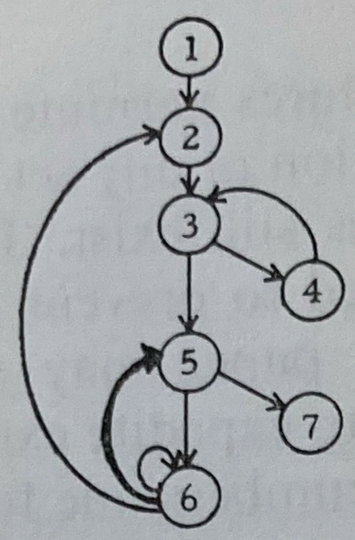
$$R_i \cap R_j = \text{NULL}$$

or

$$R_i \cap R_j = R_i, \quad \text{i.e. } R_j \text{ covers } R_i.$$

EXAMPLE 1. The flow graph  gives the region list

3
2–3–4

EXAMPLE 2. The flow graph  gives the region list

6
5–6
2–3–4–5–6

Note that in example 2, 3–4 is strongly connected and 2–3–5–6 is also strongly connected but having them both on the region list would violate condition 3. During the construction of the list one of them, say 3–4, may temporarily be placed on the list but when 2–3–5–6 is found then neither of them can remain on the list. Their union, 2–3–4–5–6, can, however, remain on the list in this example.

The appendices contain further examples.

The optimizer uses the region list as follows:

1. The regions are sequentially selected from the list. Thus when region R_i is being optimized, regions $R_1, R_2, \ldots, R_{i-1}$ have already been optimized.
2. If the region to be optimized, R_i, covers any of the regions already examined, only the unexamined blocks in R_i need optimization. The total region, however, is used to determine definitions and uses of variables etc. The set, R_i', of unexamined blocks in R_i is determined by:

$$R_i' = R_i \cap \left(\bigcap_{j=1}^{i-1} \bar{R}_j \right).$$

3. After the optimization of a region all of the non-redundant instructions removed from the region are inserted into the blocks immediately preceding or succeeding the region or formed into new blocks and put on the entry and exit paths of the region.
4. When new blocks are generated by the optimization of region R_i, the entries $R_{i+1}, R_{i+2}, \ldots, R_n$ on the region list may be modified to reflect the existence of the new blocks.
5. The blocks generated by the optimization of one region are indistinguishable from the original blocks when covering regions

are optimized. This causes a "bubbling" of code: as successive regions are examined instructions may be moved several times—further and further away from their initial position in the program.

In example 2 on page 244, $R_i : 6$ is examined first. If it produces a new block, block 8, to be inserted between blocks 5 and 6, the region list becomes

$R_1 : 6$
$R_2 : 5–6–8$
$R_3 : 2–3–4–5–6–8$

The blocks 5 and 8 of R_2 are now examined. Suppose this creates a new block, block 9, to be put between 3 and 5. The region list would become

$R_1 : 6$
$R_2 : 5–6–8$
$R_3 : 2–3–4–5–6–8–9$

Finally blocks 2–3–4–9 are examined and the code removed from them would go into block 1 (since block 1 has only 1 successor). By this process instructions in block 6 might end up in block 1. The created blocks might become vacuous.

Some of the advantages of the region list should now be apparent. Others are:

1. The program is partitioned. If total optimization is not desired the process can be halted after optimizing any region. If frequency information is available it can be used to select the regions and even those blocks in the region for optimization.
2. The optimization techniques developed for regions require very little information about the environment of a given region—the blocks not in the region and the total program flow.
3. Inter-region information is minimized. The main exception is the decrementability table described later.

The procedure for developing the region list is now given.

Region list construction

Given a program whose basic blocks have been arbitrarily numbered 1, 2, . . ., n, the region list is developed:

1. A n × n Boolean connectivity matrix, C, is constructed. $C_{ij} = 1$ if block j is an immediate successor of block i; it is zero otherwise.

The connectivity matrix for example 2 on page 244 is:

C	1	2	3	4	5	6	7
1	0	1	0	0	0	0	0
2	0	0	1	0	0	0	0
3	0	0	0	1	1	0	0
4	0	0	1	0	0	0	0
5	0	0	0	0	0	1	1
6	0	1	0	0	1	1	0
7	0	0	0	0	0	0	0

(*Programming note:* The optimizer was written to run on the 7094 and all binary data is packed in 35 bits per word—the sign bit is not used. Usually the binary data represents blocks: bit 5 of word 1 is for block 5, bit 5 of word 2 for block 40. Such a set of n consecutive bits is referred to as a *Boolean vector* in this paper. The C matrix, therefore, consists of n Boolean vectors.)

2. A list, P, of unique closed paths in the program is now developed. Each entry on the list is a Boolean vector expressing the blocks on a closed path: $P_{ij} = 1$ if block j is on a closed path which happens to be the ith entry on the P list. P is developed as follows:

(a) If $C_{ii} = 1$ then block i is an immediate successor of itself and a closed path exists. An entry is made on the P list. For the example the P list would now have one entry:

	1	2	3	4	5	6	7
P_1:	0	0	0	0	0	1	0

(b) Successive powers, l, $(1 < l \leq n)$ are taken. $C^l_{ij} = 1$ if there is a path of length l from i to j. The length of the path is the number of edges (not necessarily unique) traversed. The reader is referred to ref. [3] for more information on connectivity matrices.

A reasonably fast method of raising C^l to C^{l+1} can be developed: C^{l+1} is initialized to zero. Then, for each entry, i, in the flow table and for each immediate successor, j, to i, $C^{l+1}_i = C^{l+1}_i \vee C^l_j$ is formed. Since i and j are expressed as integer indices in the table, the selection of rows of C^l and C^{l+1} for ORing is fast. The number of operations involved is a linear function of the number of branches in the program.

If $C^l_{ii} = 1$ then there is a closed path of length l containing i. Note that if $C^l_{ii} = 1$ then C^{l+1}_{ii}, C^{l+1+1}_{ii}, etc., will also be 1.

Furthermore if block j is on the closed path with i, then $C^l_{jj} = 1$.

(c) In order to separate out the closed paths imbedded in a given C^l, an integer distance matrix, D, is kept. D_{ij} contains the length of the shortest path from block i to block j. Whenever $C^l_{ij} = 1$ and $D_{ij} = 0$, D_{ij} is set to 1.

(d) A possible closed path, PP, is constructed whenever $C^l_{ii} = 1$ by

1. Setting $PP_i = 1$ and
2. For all $j \neq i (1 \leq j \leq n)$ if $D_{ij} \neq 0$ and $D_{ji} \neq 0$ and $D_{ij} + D_{ji} \leq 1$, $PP_j = 1$.

(e) PP is added to the P list if it is unique and if it does not have a strong tendency to exit. This latter is determined from the available frequency information.

3. Using the list, P, of closed paths the list of regions, R, is constructed.

Articulation Points

The graph theoretic concept of *articulation points* [2] is useful when invariant instructions are removed from a region. A block in a region is an articulation block if the block lies on every entry–exit path.

The algorithm for constructing the Boolean vector, A, of articulation blocks in region R_k is

1. The entry blocks of the region are located:

(a) The Boolean vector, PV, of the immediate predecessors of all the blocks in R_k is formed by:

$$PV = \vee\{C_{-1} | i \in R_k\}$$

i.e. PV is the union of the columns of C for blocks in the region.

(b) The Boolean vector PVR of blocks which immediately precede the region is obtained by:

$$PVR = PV \wedge \overline{R}_k$$

(c) The entry blocks of the region are expressed in Boolean vector IN where

$$IN = [\vee\{C_i | i \in PVR\}] \wedge R_k.$$

2. The exit blocks, if a region can be expressed by a Boolean vector, XN, which is similarly constructed:

(a) $$SV = \vee(C_i | i \in R_k)$$

(b) $$SVR = SV \wedge \overline{R}_k$$

(c) $$XN = [\vee\{C_{-i} | i \in SVR\}] \wedge R_k$$

3. If IN has only 1 entry then it must be an articulation block. If XN has only 1 entry then it must be an articulation block. If they each have only 1 entry and the entries are the same then that is the only articulation point for the region.

4. In general for each block $i \in R_k$ an *interference matrix* C^* is constructed from C by zeroing the ith row and the ith column of C and all rows and columns of C which are not in R_k and then raising this modified Boolean matrix to the nth power. Reference 4 describes a fast method of raising a Boolean matrix to powers.

   Such an interference matrix is useful in determining whether control can pass from one block to another block without going through a given block i. By examining the interference matrix C^* constructed for i it can be determined whether or not i is an articulation block: if for all $j \in IN$, $C^*_j \wedge XN = 0$ then i must be an articulation block since there does not exist an entry–exit path without i.

Data Dependencies

Intra-block dependencies

Although the instructions in a block are usually considered to have a linear ordering because they will be picked up sequentially on a single instruction counter machine, they have in fact only a partial ordering dictated by the data dependency relationships. This can be depicted by a *dependency tree* in which a node represents the result of an operation and each input link is an operand for the instruction. Nodes having no input links are essentially fetches of operands not generated in the block.

The *level number* or *depth* of an instruction in a block can be defined in terms of the dependency tree for the block. A node having no input links has level number zero. The level number of a node is n where n is the maximum number of links along any path between that node and the zero level nodes.